

毕业设计答辩讲稿

2026-05-25

毕业设计答辩讲稿参照

本文档对应当前 ppt/finalterm/2022112879-郑彦文-毕业设计答辩.pptx。当前 PPT 共 18 页，其中第 1-13 页为正式汇报，第 14-17 页为备份页，第 18 页为结束页。正式汇报建议控制在 8-10 分钟，目标时长约 9 分钟 20 秒，问答约 5 分钟。

使用时不要逐字背诵。每页只抓住一个问题：本页回答什么、屏幕上看哪里、我想让老师记住哪一句、如果被追问再补充什么。

一、整体主线

本次答辩的主线是：

高代价、可失败的蛋白质设计工具链，如何在执行过程中做可解释、可恢复、可审计的规划与恢复决策。

需要让老师记住四句话：

1. 本文贡献集中在运行时工作流规划，不是训练新的蛋白质生成模型。
2. CEBRA-WP 先用 ToolKG、schema、I/O、安全和预算做硬过滤，再对合法候选做评分和重排。
3. 运行时自适应层用 StepResult、SafetyResult、失败历史和预算进度更新 Lite belief-state。
4. 算法输出 continue、patch_local、suffix_replan、stop 四类动作，并通过 FSM/HITL、EventLog 和 Snapshot 进入可审计流程。

二、时间分配

页码	页面主题	建议时间	讲清楚的事
1	封面	10 秒	题目与工作定位
2	汇报结构与核心问题	30 秒	本文围绕一个运行时控制问题展开
3	为什么需要 workflow 规划机制	45 秒	工具链长、贵、会失败，需要控制层
4	系统架构如何支撑算法落地	45 秒	架构简讲，补充五层职责与算法位置
5	算法接口链	40 秒	模块之间传结构化对象，不传自然语言计划
6	FSM/HITL 约束	40 秒	算法动作不直接越权执行
7	CEBRA-WP 总览	50 秒	静态规划层 + 运行时自适应层
8	静态规划	55 秒	候选生成、硬过滤、静态评分
9	Lite belief-state	55 秒	稀疏观测如何变成五维状态
10	动态决策	60 秒	有界重排和四类恢复动作
11	算法落到代码	45 秒	理论对象对应实现字段和模块
12	验证	55 秒	系统测试与 84-run 消融结论
13	贡献与边界	35 秒	收束贡献，不夸大结论

备份页第 14-17 页只在问答时使用，不主动展开。

三、逐页讲稿

第 1 页：封面

回答的问题：这项毕业设计做的是是什么。

建议口播：

各位老师好，我是软件工程专业郑彦文。我的毕业设计题目是《基于大模型驱动的 Agent 协作新一代蛋白质设计系统开发》。这项工作关注的不是重新训练一个蛋白质生成模型，而是面向已有蛋白质设计工具链，构建一个可规划、可执行、可恢复、可审计的工作流控制系统。

转场：

下面我先说明整场汇报围绕的核心问题。

第 2 页：汇报结构与核心问题

回答的问题：本次答辩到底围绕哪条线展开。

屏幕重点：中间五个阶段，尤其是蓝色的“核心算法”。

建议口播：

本次答辩围绕一个问题展开：高代价、可失败的蛋白质设计工具链，如何在执行过程中做可解释的规划与恢复决策。前两部分先说明工程约束和系统支撑，中间重点讲核心算法 CEBRA-WP，后面再说明它如何落到代码，以及用哪些测试和消融实验验证。

这里的核心判断是，本文的贡献集中在运行时工作流规划：先约束候选可执行，再依据运行时证据选择 `continue`、`patch_local`、`suffix_replan` 或 `stop`。

补充说明：

这页不要展开算法细节，只给老师建立“后面主要看算法机制和工程落地”的预期。

转场：

先看为什么蛋白质设计场景需要这样的工作流规划机制。

第 3 页：为什么需要 workflow 规划机制

回答的问题：为什么不能只把几个工具顺序调用起来。

屏幕重点：接口异构、高代价调用、失败可恢复、决策需审查。

建议口播：

蛋白质设计工具链不是一次模型调用，而是生成、结构预测、质量门禁、目标评分和结果汇总组成的长链路。工程上有四个问题。第一，序列、结构、评分工具的输入输出格式差异明显。第二，结构预测和重型打分属于高代价调用，证据不足时不应该盲目执行。第三，参数错误、I/O 缺失、质量门禁失败都可能出现，但这些失败不一定意味着任务必须终止。第四，高风险、高成本或不确定的决策需要人工确认。

本文的边界也在这页说明清楚：我不训练新的蛋白质生成模型，不宣称湿实验验证，也不替代 AlphaFold、ESMFold 等底层工具。我的切入点是在已有工具之上实现 workflow 控制层。

可以补充的难点：

难点不在于把工具简单串起来，而在于失败后如何判断：是继续、局部修补、后缀重规划，还是受保护地停止。这个判断必须同时考虑工具约束、证据、成本和安全边界。

转场：

有了问题边界，下面看系统架构如何承载这个控制层。

第 4 页：系统架构如何支撑算法落地

回答的问题：CEBRA-WP 在系统中处于什么位置，为什么不会越权执行。

屏幕重点：左侧系统架构图只看层次，右侧看“算法位置”和“架构约束”。

建议口播：

这页 PPT 对体系架构做了简化，图上重点看层次关系。系统可以理解为五层：最上层是输入交互层，承接 Web、CLI、API 和人工决策；第二层是智能规划层，包含 PlannerAgent、ProteinToolKG 和 CEBRA-WP；第三层是 workflow 执行层，包含 ExecutorAgent、PlanRunner 和 StepRunner；第四层是安全与汇总层，由 SafetyAgent 输出风险信号，SummarizerAgent 汇总结果；最底层是资源与证据层，包括 ToolAdapter、KG、EventLog、Snapshot 和产物。

CEBRA-WP 位于智能规划层，它负责生成候选、过滤、评分和运行时重排，但它的输出不会直接调用工具。真正执行必须经过 workflow 执行层、FSM/HITL 和证据层约束。

可以补充的难点：

这里的工程难点是把大模型规划的不确定性限制在结构化对象里。Planner 只产出候选计划，不直接执行；Executor 只执行已确认步骤，不越过状态机；Safety 只输出风险信号，不直接改计划；Summarizer 只汇总已有证据，不重新计算。这种职责隔离是系统可信性的基础。

转场：

架构分层之后，还要说明模块之间到底传递什么信息。

第 5 页：算法接口链：模块之间传递什么

回答的问题：模块之间如何传递信息，讲到哪一步即可。

屏幕重点：上方规划与确认链，下方执行结果、运行时状态与恢复动作链。

建议口播：

模块之间的信息传递可以理解为算法接口链。系统不是让 Agent 直接互相传自然语言计划，而是把算法所需信息收束成结构化对象。

上方这条链是规划侧：任务先被表示为 TaskSpec，包含目标、约束和预算；Planner 生成 CandidateSet，包括 Plan、Patch 和 Replan 候选；候选经过 Feasibility 与 Score，得到 blocked_by 和 score_breakdown；需要确认的候选进入 PendingAction，最后形成 Decision。

下方这条链是运行时侧：Executor 执行后产生 StepResult，SafetyAgent 产生 SafetyResult，这些观测汇入 RuntimeState；RuntimeEvaluator 再输出 RecoveryAction，最后通过 EventLog 和 Snapshot 留下审计与恢复证据。

可以补充的难点：

难点在于结构化契约要同时服务三件事：让算法能计算，让执行器能运行，让日志和快照能回放。自然语言解释可以辅助展示，但不能作为模块间的真实控制接口。

转场：

有了结构化对象，还需要状态机约束算法动作，避免恢复动作越权。

第 6 页：FSM/HITL：恢复动作如何被约束

回答的问题：算法输出动作后，系统如何保证它可审查、可追溯。

屏幕重点：左侧状态迁移图中的等待态，右侧三类说明框。

建议口播：

这页说明 CEBRA-WP 的输出如何被 FSM 和 HITL 约束。图中的虚线区域对应人工确认边界。系统中有几类关键等待态：初始计划确认、局部修补确认、重规划或 stop 候选确认。进入等待态后，Executor 不继续调用工具，而是等待 PendingAction 被确认。

算法输出 continue、patch_local、suffix_replan 和 stop 后，并不是直接改变底层工具执行，而是转成 Decision、EventLog 和 Snapshot。这样每个恢复动作都可以被审查、记录和恢复。

可以补充的难点：

尤其是 stop 动作不能由算法静默执行，因为它代表放弃当前任务或进入终止路径。本文把 stop 作为 terminal_stop 类型候选进入 replan_confirm / HITL，保证高风险动作有人工确认边界。

转场：

到这里，系统边界已经说明清楚。下面进入核心算法 CEBRA-WP。

第 7 页：CEBRA-WP：静态规划层 + 运行时自适应层

回答的问题：CEBRA-WP 到底是什么。

屏幕重点：上方四步流程和下方“输入、核心状态、输出”。

建议口播：

CEBRA-WP 可以分成两层。第一层是静态规划层，它在执行前根据任务目标、约束和工具知识图谱生成候选，做硬可行性过滤，并建立静态评分。第二层是运行时自适应层，它在执行过程中读取 StepResult、SafetyResult、失败历史和预算信息，更新 Lite belief-state，再对候选和恢复动作做有界调整。

这套机制的输入包括 Task、Constraints、ToolKG、runtime context 和 policy；核心状态是四维运行时状态；输出是 continue、patch_local、suffix_replan 和 stop。

可以补充的难点：

这里最重要的边界是：运行时状态只修正已经合法的候选，不覆盖 tool、schema、I/O、safety 和 budget 的硬约束。也就是说，运行时证据可以改变优先级，但不能把不可执行候选重新放回执行队列。

转场：

先看执行前的静态规划层如何保证候选可执行。

第 8 页：静态规划：候选、硬过滤与静态评分

回答的问题：执行前如何保证候选工具链可执行，并建立先验排序。

屏幕重点：上方五步链路和底部 S_static。

建议口播：

静态规划层先回答“哪些候选能执行”，再回答“哪条链先验上更值得尝试”。输入是任务目标、预算、允许或禁用工具，以及 ProteinToolKG 中的工具能力和 I/O 信息。候选生成后，系统先做 hard feasibility，检查工具是否存在、schema 是否匹配、跨步骤 I/O 是否闭合、是否违反 safety 或 budget。

通过硬过滤后，系统再用 objective、cost、risk、recovery complexity 和 reliability 等因素形成静态评分 S_static，并保留 Top-K 多样候选。这样做可以避免把不可执行候选带入运行时阶段，也避免一次性押注在单条 LLM 生成链路上。

可以补充的难点：

LLM 生成的候选可能“看起来合理”，但实际上 schema 不对、输入输出接不上，或者需要的工具不可用。因此硬过滤必须是确定性的，安全和预算也不能被高目标分抵消。静态评分只在候选合法之后生效。

转场：

静态规划解决执行前的问题，执行过程中还需要处理稀疏、不完整的运行时观测。

第 9 页：Lite belief-state：把稀疏观测压缩成可解释状态

回答的问题：执行中的失败、风险和成本信息如何被算法使用。

屏幕重点：左侧五类观测，中间 Lite belief-state，右侧五个状态变量。

建议口播：

运行时阶段的难点是观测不完整。一次工具失败不一定说明整条路线都错了，一次成功也不能说明后续一定可靠。因此系统没有把运行时事件直接变成动作，而是先更新一个 Lite belief-state。

这个状态来自五类信息：StepResult 的输出、指标和错误；SafetyResult 的风险标记；失败上下文和 patch/replan 历史；预算与任务进度；以及 HITL Decision。它被压缩成五个变量：p_success 表示完成任务的代理概率，p_structural_failure 表示结构性失败压力，recovery_margin 表示继续恢复的余量，expected_remaining_cost 表示预期剩余成本，evidence_sufficiency 表示证据是否足以进入高代价步骤。

可以补充的难点：

这不是完整 POMDP，也不是学习得到的最优策略，而是受 belief-state planning 启发的工程化低维状态代理。当前实现采用确定性更新规则表：成功提高 s/r，失败提高 f/c，safety block 作为强负证据。这样做的好处是可解释、可序列化、能进入 Snapshot，也能在实验中观察到。

转场：

有了运行时状态，下一步是用它影响候选重排和恢复动作选择。

第 10 页：动态决策：重排候选并选择恢复动作

回答的问题：运行时状态如何影响下一步动作。

屏幕重点：上方 U_{pi} 和 ActionUtility，下方四类动作。

建议口播：

动态阶段有两个输出。第一个是候选重排：静态分或后验目标分作为基础，运行时状态只做有界修正，也就是 $U_{pi} = \text{clip}(S_{post} + \Delta, 0, 1)$ 。这里的 Δ 不会改变硬过滤结果，只影响已通过硬过滤候选的排序。

第二个输出是恢复动作选择。ActionUtility 会根据当前状态、预算、恢复余量和候选形状，在四类动作中选择：成功概率和证据充分度尚可时 continue；故障局部化且恢复余量足时 patch_local；结构性失败压力高、当前后缀不可靠时 suffix_replan；成功率低、预算压力高时 stop，但 stop 会受保护进入 HITL。

可以补充的难点：

运行时信号容易被单次失败误导，所以 Δ 必须有界；高代价调用不能盲目重跑，所以 budget pressure 和 evidence_sufficiency 会抑制无效调用；stop 风险最高，所以实现中它不直接终止任务，而是进入 replan_confirm / HITL。

转场：

算法讲清楚之后，还要证明它不是停留在概念图里，而是落到了代码对象和模块中。

第 11 页：算法对象如何落到代码

回答的问题：算法对象在实现里对应什么字段和模块。

屏幕重点：表格中的三列：算法对象、实现字段/模块、作用。

建议口播：

这一页用来说明算法已经工程落地。硬可行性过滤落在 `candidate_feasibility` 和候选生成器中，用来阻断 `tool`、`schema`、`I/O`、`safety`、`budget` 违规候选。静态评分落为 `score_breakdown.overall` 和 `static_score`。后验目标落为 `posterior_objective`。Lite belief-state 落为 `RuntimeState` 和 `belief_state.py`。运行时修正由 `runtime_adjustment` 和 `RuntimeEvaluator` 完成。动作效用由 `ActionUtility` 计算。

最后，算法结果不会绕过系统直接执行，而是进入 `PendingAction`、`Decision`、`EventLog` 和 `Snapshot`。这也是它可审计、可恢复的实现基础。

可以补充的架构说明：

如果老师追问代码结构，可以补充：API 层负责暴露任务、事件、报告和人工决策接口；`workflow` 层负责 FSM、`PlanRunner`、`StepRunner` 和 `RuntimeEvaluator`；`adapters` 层封装本地或远程工具；`storage` 层保存事件日志和快照。前端只展示后端状态和决策入口，不自行推导状态机。

转场：

实现之后，需要用测试和实验说明机制是否真的可运行、可观察。

第 12 页：验证：系统测试 + 84-run 消融

回答的问题：如何验证系统有效，但不过度夸大。

屏幕重点：上方五个数字，右侧 84-run 机制证据。

建议口播：

验证分两部分。第一部分是系统测试，共 13 个测试用例，12 个通过，1 个 CLI 部分通过，覆盖 API、任务录入、HITL/FSM/快照、Web/CLI、端到端工具链，以及异常、安全和恢复路径。第二部分是 84-run 消融实验，总体 81/84 进入 DONE 终态，lite 组 21/21 产生 RuntimeState，说明运行时状态可以被记录和观察。

从高代价调用看，fixed 高代价调用是 28 次，dynamic 和 lite 都是 20 次，相比 fixed 减少 28.6%。这说明动态机制在减少无效高代价调用方面有证据。

结论边界：

这组实验没有证明 CEBRA-WP 显著提升成功率，所以本文不把成功率提升作为主要结论。更稳妥的结论是：机制可执行、状态可观测、恢复可解释，并且成本控制有实验支持。

转场：

最后总结本文贡献和边界。

第 13 页：贡献、局限与结论边界

回答的问题：本文最后能稳妥地声称什么。

屏幕重点：四项贡献和底部结论边界。

建议口播：

本文贡献可以概括为四点。第一，实现了一个可恢复、可审计的多 Agent 蛋白质设计 workflow 原型，打通任务接入、候选计划、工具执行、人工决策和报告输出。第二，提出并实现 CEBRA-WP，将约束、证据、轻量状态和恢复动作纳入规划闭环。第三，用 FSM、HITL、Decision、Snapshot 和 EventLog 约束 Agent 行为。第四，用系统测试和 84-run 消融实验支撑主要结论。

同时，本文的结论限定在工作流层：验证的是规划、执行、恢复和审计机制，不把计算评分扩大为湿实验结论，也不声称 CEBRA-WP 显著提升成功率。候选蛋白的生物学有效性需要后续实验验证。

结束句：

我的汇报到这里结束，恳请各位老师批评指正。

四、备份页使用方式

第 14 页：备份：公式与字段映射

使用场景：老师追问 S_{static} 、 Δ 、 U_{pi} 、 U_a 等符号具体对应什么。

回答要点：

- F_h 对应 `candidate_feasibility`，表示硬可行性过滤。
- S_{static} 对应 `score_breakdown.overall`，表示静态候选效用。
- x_t 对应 `RuntimeState`，表示多维运行时状态。
- Δ 对应 `runtime_adjustment`，表示有界运行时修正。
- U_a 对应 `ActionUtility`，表示恢复动作效用。

口头补充：

正式页没有展开复杂公式，是因为本文实现采用可解释、可测试的确定性规则和字段映射；备份页用于说明每个理论对象在代码中有对应位置。

第 15 页：备份：ProteinToolKG 如何约束工具链

使用场景：老师追问 ToolKG 的作用，或者问硬过滤依据从哪里来。

回答要点：

- ProteinToolKG 记录工具能力、输入输出、成本、风险和可用性。
- 实线表示 I/O 兼容链，虚线表示 fallback 或 degraded readiness。
- CEBRA-WP 的 hard feasibility 会使用 tool、schema、I/O、cost/risk 等信息过滤不可执行候选。

口头补充：

这张图不需要逐个节点讲，只要说明：工具知识图谱把“能不能接上、能不能执行、代价和风险如何”变成确定性约束，避免 LLM 生成看似合理但不可运行的链路。

第 16 页：备份：失败样本与恢复边界

使用场景：老师追问 3 个 FAILED run 说明什么，或者问系统是不是能恢复所有失败。

回答要点：

- 失败样本没有被隐藏，而是保留 event log 和 snapshot。
- fixed_threshold_gate 可以触发 patch，但可能出现循环耗尽。
- lite belief-state 能让 RuntimeState 可观测，但不保证每次都打破 WAITING_PATCH 循环。
- dynamic_no_belief_state 可以在执行前阻断不可执行候选。

口头补充：

这页说明机制边界，而不是证明所有失败都能恢复。本文更谨慎的结论是：失败路径可记录、可解释、可追溯，部分恢复动作可受控进入 HITL。

第 17 页：备份：测试证据索引

使用场景：老师追问测试材料和证据在哪里。

回答要点：

- EVD-API 覆盖健康检查、readiness、任务详情、事件、报告和 pending action。
- EVD-TEST 覆盖 API/Web、FSM/HITL/快照、安全恢复工具和 CLI 日志。
- FIG-SV 包含 Dashboard、Task Builder、Task Detail、Timeline 截图。
- EVD-LOG 包含 EventLog、Snapshot、Report、terminal_stop 和等待态样本。
- EVD-EXP 包含 smoke / clean run 与 84-run 实验矩阵聚合产物。

口头补充：

证据链主要回答两个问题：系统测试是否覆盖关键路径，运行时恢复决策是否可追溯。

五、常见问题准备

1. 本文的算法贡献是什么？

建议回答：

CEBRA-WP 是面向高代价蛋白质设计工作流的约束化、证据感知、轻量状态引导、恢复自适应规划机制。它的贡献不在底层蛋白生成，而在运行时 workflow 规划：先做硬可行性过滤，再用静态评分建立先验排序，最后用运行时状态对合法候选和恢复动作做有界调整。

2. CEBRA-WP 和普通 planner 有什么区别？

建议回答：

普通 planner 更偏向一次性生成计划。CEBRA-WP 在计划候选之外加入 hard feasibility、score_breakdown、Lite belief-state、ActionUtility 和 FSM/HITL 审计入口。它能处理执行中的失败、风险和预算变化，而不是只给出初始流程。

3. 为什么不用完整 POMDP、RL 或学习型策略？

建议回答：

本项目的数据量和实验条件不足以支持稳定学习策略，而且毕业设计重点是可解释、可验证的工程机制。因此本文采用 Lite belief-state：低维、确定性、可序列化、可写入 Snapshot。它不是完整 POMDP，也不声称学习到了最优 policy。

4. 硬过滤和评分的边界是什么？

建议回答：

硬过滤处理不可违反的条件，例如工具是否存在、schema 是否合法、I/O 是否闭合、安全是否阻断、硬预算是否越界。评分只在候选通过硬过滤之后生效，用于比较目标匹配、成本、风险、恢复复杂度和可靠性。安全和预算不能被高分抵消。

5. 模块之间如何传递信息？

建议回答：

通过结构化对象传递，不依赖自然语言计划。规划侧是 TaskSpec、CandidateSet、candidate_feasibility 和 score_breakdown；执行侧回传 StepResult、SafetyResult 和 failure_context；决策侧留下 PendingAction、Decision、EventLog 和 Snapshot。

6. 为什么实验里不强调成功率提升？

建议回答：

84-run 结果没有证明 CEBRA-WP 显著提升成功率，所以本文不把成功率作为主要结论。本文强调的是机制层面的证据：lite 组 21/21 产生 RuntimeState，说明状态可观测；dynamic/lite 相比 fixed 减少高代价调用，说明成本控制有证据；EventLog 和 Snapshot 说明恢复路径可审计。

7. 系统如何保证恢复动作可审计？

建议回答：

恢复动作不会直接调用工具，而是进入 PendingAction 和 Decision。状态迁移由 FSM 约束，EventLog 记录事件，Snapshot 保存上下文。系统中断后可以恢复到等待态或已确认状态，避免 Agent 私自跳过人工确认。

8. 本人工作量如何体现？

建议回答：

不需要单独强调“独自完成”，可以从四个层面体现：一是系统架构和多 Agent 职责边界；二是 CEBRA-WP 的候选过滤、运行时状态和动作选择机制；三是 API、workflow、adapter、KG、storage 和 Web 工作台的工程实现；四是系统测试和 84-run 消融实验的验证证据。

六、答辩时避免的表述

- 不说“本文证明了候选蛋白具有湿实验有效性”。
- 不说“CEBRA-WP 显著提升成功率”。
- 不说“Lite belief-state 是完整 POMDP”。
- 不说“posterior objective 等价于真实生物学目标”。
- 不把前端展示说成核心贡献，前端只是状态、决策和证据链的可视化入口。